The University of Azad Jammu & Kashmir, Muzaffarabad

Department of Software Engineering

Software Construction & Development (Lab)

Lab 03

Task Title:	**Sequence Modeling & Interaction Analysis**
Submitted to:	**Engr. Dr. Asma Javed**
Student Name:	**Feeza Kulsoom**
Roll No:	**2024-SE-03**
Reg Number:	**2024-UMDB-004738**

Task 1: ATM Cash Withdrawal System

Problem Statement: Draw a sequence diagram that models the data processing involved when a customer withdraws cash from an Automated Teller Machine (ATM), capturing object lifelines, sequential message passing, alternative execution paths (opt/alt fragments), and physical hardware actuation.

1.1 System Lifelines & Object Identification

- **Customer (Actor):** The account holder who initiates the physical card insertion, enters their secret PIN, specifies withdrawal parameters, collects dispensed currency, and retrieves the payment receipt.
- **ATM Screen / User Interface:** The front-end boundary interface presenting visual prompts, keypad input options, numeric amount selections, and instructional display messages.
- **ATM Controller / Core Processing Unit:** The central control subsystem orchestrating workflow logic, coordinating input verification, dispatching transaction payloads to the bank server, and signaling hardware modules.
- **Bank Server / Central Banking Host:** The authoritative external financial backend verifying customer credentials, checking ledger balances against account limits, logging the debit transaction, and returning transactional authorization tokens.
- **Cash Dispenser:** The electromechanical internal hardware peripheral that counts, verifies, and physically discharges banknotes to the output cash tray.
- **Receipt Printer:** The dedicated thermal printer peripheral generating printed audit receipts verifying transaction amounts, timestamp, terminal ID, and remaining balance.

1.2 Step-by-Step Interaction Flow & Message Exchange

1. **Card Insertion & PIN Prompt:** The Customer inserts their debit card into the card reader. The ATM Controller detects the card, reads the EMV chip data, and triggers the ATM UI to display 'Enter PIN'.
2. **PIN Entry & Verification:** The Customer inputs their secret 4-digit PIN via the keypad. The ATM Controller forwards an encrypted credential packet to the Bank Server (verifyPIN(cardNumber, pin)).

3. **PIN Validation Response:** The Bank Server verifies the encrypted hash against database records and responds with pinValidStatus (Success or Invalid PIN).
4. **Account Selection & Amount Entry:** Upon successful authorization, the ATM UI displays the transaction menu. The Customer selects 'Fast Cash' or inputs a custom withdrawal amount.
5. **Fund & Daily Limit Validation:** The ATM Controller forwards a withdrawal request (requestWithdrawal(accNum, amount)) to the Bank Server. The Bank Server validates that the account balance >= requested amount and that daily withdrawal velocity limits are not exceeded.
6. **Account Ledger Debit:** The Bank Server commits the debit transaction (debitAccount(amount)), deducts the ledger balance, and returns transactionAuthorized(authCode, remainingBalance).
7. **Hardware Dispense Signal:** The ATM Controller signals the Cash Dispenser via dispenseCash(amount). The dispenser mechanical rollers deliver banknotes to the collection slot.
8. **Receipt Printing & Card Return:** The Controller triggers the Receipt Printer via printReceipt(transDetails, remainingBalance) and ejects the debit card via ejectCard().
9. **Customer Collection:** The Customer takes the dispensed banknotes, collects their printed receipt, and retrieves their card to terminate the session.

1.3 Sequence Diagram

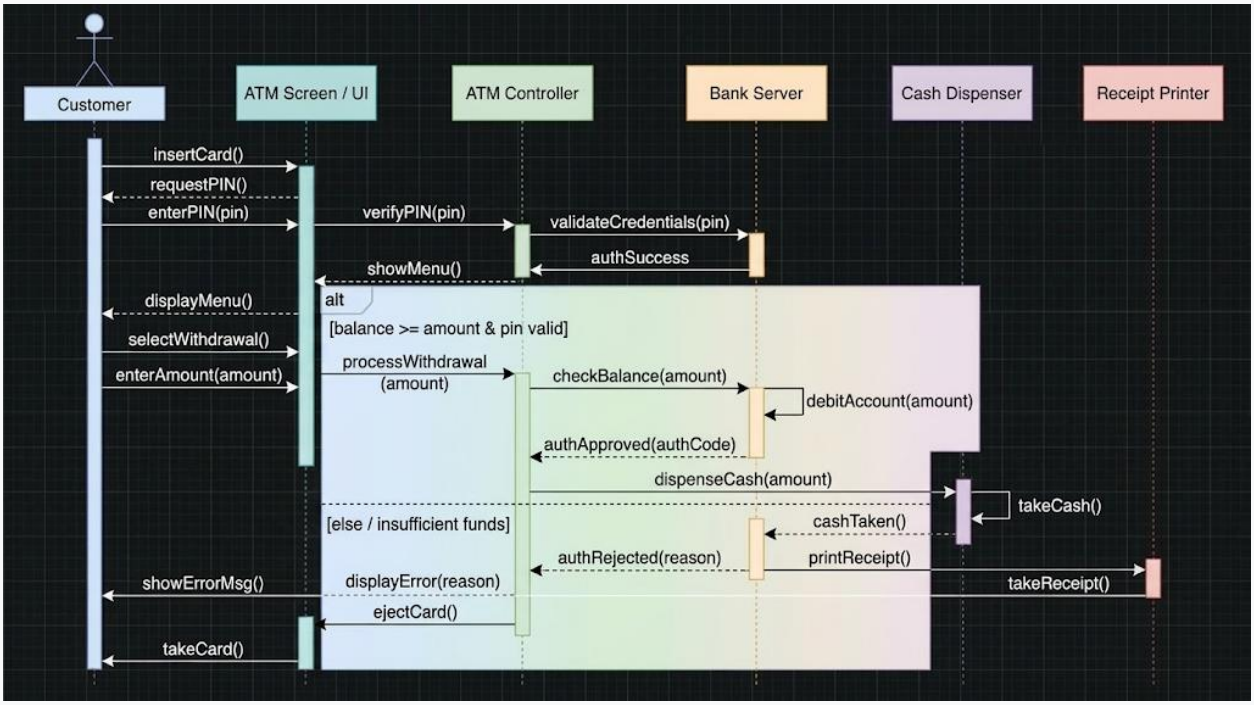


Figure 1.1: Sequence Diagram – ATM Cash Withdrawal Processing Flow

Task 2: Online Ticket Reservation System

Problem Statement: Develop a comprehensive UML Sequence Diagram for the Online Ticket Reservation System (building directly on the domain model established in Lab 01). Model the temporal interactions between the Customer, Web Client UI, Reservation Controller, Inventory/Seat Database, External Payment Gateway, and Notification/E-Ticket Service.

2.1 System Lifelines & Entity Identification

- **Passenger / Customer (Actor):** The traveler who queries trip schedules, selects specific passenger seats, submits billing details, authorizes online payment, and receives the confirmed e-ticket.
- **Web Client / Frontend UI:** The browser-based interface rendering search filters, dynamic seat maps, booking review summaries, and payment forms.
- **Reservation System / Backend Controller:** The core business application server managing session state, processing reservation logic, orchestrating temporary seat locking, and calling third-party gateways.
- **Inventory / Schedule Database:** The relational persistent storage engine containing route timetables, real-time seat availability matrices, coach layouts, fare classes, and customer reservation logs.
- **Payment Gateway (External Service):** The PCI-DSS certified payment clearinghouse that authenticates credit card/banking credentials, executes multi-factor authorization, and returns monetary charge settlements.
- **Notification & Ticketing Service:** The subsystem responsible for generating formatted PDF e-tickets equipped with scannable QR verification codes and dispatching automated confirmation emails/SMS.

2.2 Step-by-Step Interaction Flow & Message Exchange

10. **Search Trip & Route Query:** The Customer inputs origin, destination, travel date, and coach tier into Web UI. The UI invokes searchRoutes(origin, dest, date) on the Reservation Controller.
11. **Query Schedule Database:** The Controller executes queryAvailableTrips() against the Schedule Database, retrieving a dataset of available coaches, departure timings, and base fare rates.
12. **Seat Map Display & Selection:** The Customer selects a preferred route. The UI requests the real-time seat matrix via getSeatMap(tripID). The Customer clicks an open seat (e.g., Seat 14A, Executive Class).
13. **Temporary Seat Hold (Concurrency Control):** The Controller issues lockSeat(seatID, sessionTimeout=10min) to the Database to prevent double-booking collisions while the user enters passenger particulars.
14. **Checkout & Payment Delegation:** The Customer enters passenger identity information and billing credentials, clicking 'Confirm & Pay'. The Controller calls processPayment(cardInfo, amount) on the external Payment Gateway.
15. **Payment Authorization:** The Payment Gateway validates card tokens with the issuing clearinghouse, captures the payment, and returns paymentAuthorized(transID, token).
16. **Booking Finalization:** Upon verified payment, the Controller changes seat status from 'Held' to 'Reserved' in the Database via confirmBooking(seatID, pnrNumber).
17. **Ticket Generation & Notification:** The Controller signals the Notification & Ticketing Service via generateTicket(pnr, passengerDetails). The service compiles the PDF E-Ticket with QR code and dispatches it via email/SMS.

18. **UI Confirmation & Download:** The Web UI updates the client viewport displaying 'Booking Confirmed' alongside a direct button allowing the Customer to view and download their E-Ticket.

2.3 Sequence Diagram

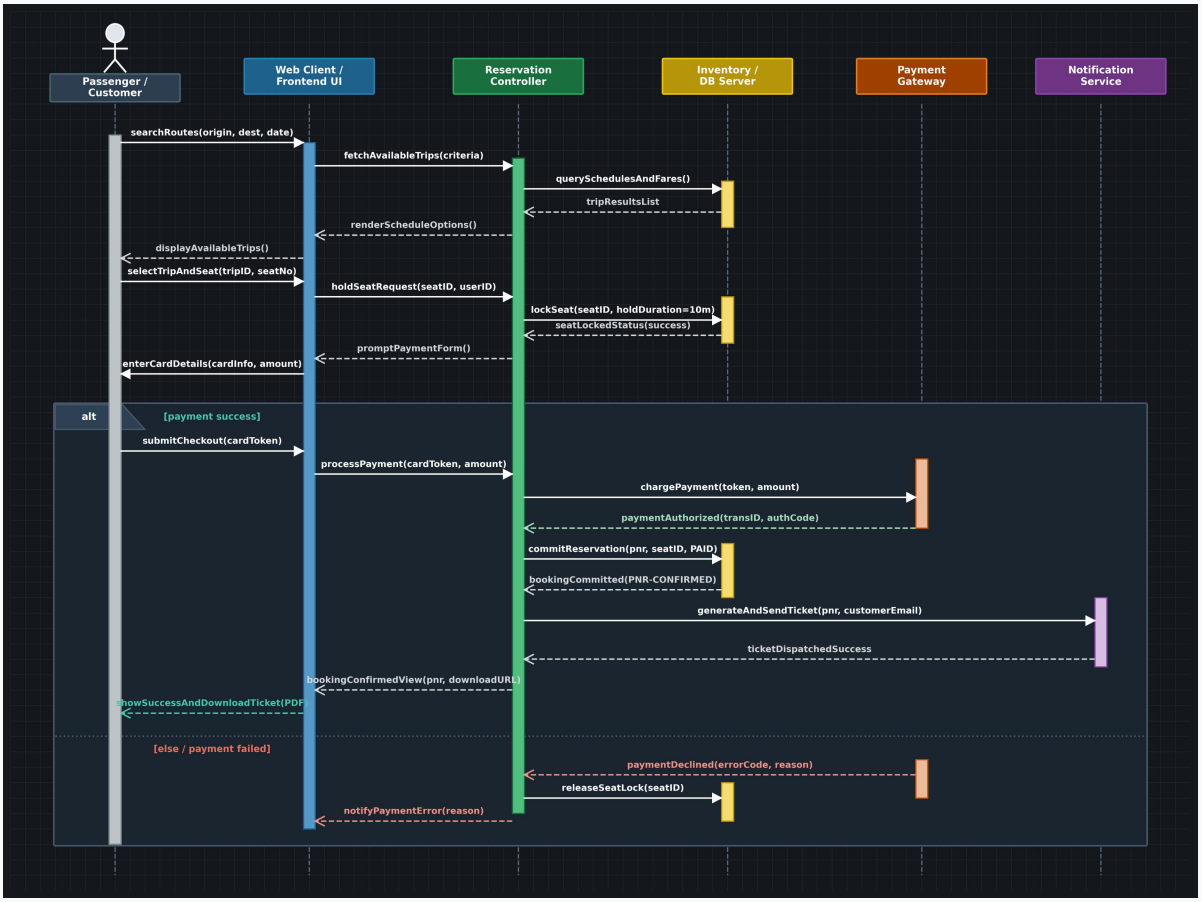


Figure 2.1: Sequence Diagram – Online Ticket Reservation & Payment Workflow